A Jose, Justin Gabriel D. Object Oriented Programming

BSIT 21

Activity # 1 - Variables

Please answer the following questions:

PART A - VARIABLE IDENTIFICATION TABLE

Variable Name	Data Type	Variable Type	Stored In
name	String	Instance Variable	Heap
grade	int	Instance Variable	Heap
n	String	Local Variable (Parameter)	Stack
g	int	Local Variable (Parameter)	Stack
s1	Student	Reference Variable	Stack
s2	Student	Reference Variable	Stack
s3	Student	Reference Variable	Stack
args	String[]	Local Variable (Parameter)	Stack

PART B - EXPLANATION

Statement	What Happens?	Memory Impact
Student s1 = new Student ("John", 85);	• The JVM calls the student constructor to initialize a new instance with the specific values "John" and 85. This process links the reference variable s1 to the newly allocated memory space containing these data fields.	• A reference variable s1 is allocated on the Stack to hold a memory address pointing to the Heap. Simultaneously, a student object is created on the Heap to store the actual instance variables name and grade.
Student s2 = new Student ("Maria", 90)	• The JVM allocates memory for a second Student instance and executes the constructor to assign "Maria" and 90 to its fields. This creates a completely independent object from s1, with the reference variable s2 holding its own unique memory address	• A new reference variable s2 is added to the Stack, while a separate Student object is created on the Heap to store the specific instance data for "Maria". This increases the total Heap usage as two distinct objects now exist simultaneously.

Statement	What Happens?	Memory Impact
Student s3 = s2;	• This statement copies the memory address stored in s2 into the new reference variable s3, meaning both variables now point to the exact same object. No new Student object is created during this assignment.	• A new reference variable s3 is added to the Stack, but the number of objects on the Heap remains unchanged. Because they share a memory address, any changes made to the object's attributes via s3 will be reflected when accessing the object through s2.
s1.grade = 95;	• The program follows the memory address stored in the reference variable s1 to locate the specific object on the Heap and updates its grade field to 95. This directly modifies the existing data within that object rather than creating a new one.	• The value stored in the grade instance variable on the Heap changes from 85 to 95, while the reference on the Stack remains unchanged. No new memory is allocated, as the operation simply overwrites data in a previously established memory location.
s3.name = "Ana";	• The program uses the memory address in s3 to locate the object on the Heap and updates its name field to "Ana." Because s3 and s2 both point to the same object, this change is visible to both reference variables.	• The name instance variable within the specific object on the Heap is updated to "Ana", while the reference pointers on the Stack remain exactly as they were. This demonstrates "aliasing" where modifying an object through one reference (s3) simultaneously changes the data seen by another reference (s2).

PART C - OUTPUT

John 95

Ana 90

Ana 90